

# Heart Disease Prediction

## MLOps Project Report

**Course:** Machine Learning Operations (MLOps) — AIMLCZG523

**Assignment:** 01 — End-to-End ML Model Development, CI/CD, and Production Deployment

**Student:** Sumankk

**Repository:** <https://github.com/Sumankk022/heart-disease-mlops>

### Table of Contents

1. Introduction & Problem Statement
2. Dataset Overview
3. Data Cleaning & EDA (Figures 1-3)
4. Feature Engineering & Modelling (Figure 4)
5. Experiment Tracking (MLflow) (Figures 5-6)
6. Model Packaging & Reproducibility
7. Testing & CI/CD Pipeline (Figure 7)
8. Model Containerization (Docker) (Figures 8-9)
9. Production Deployment (Kubernetes & Helm)
10. Monitoring & Logging
11. Architecture & Data Flow
12. How to Reproduce
13. Conclusion

## 1. Introduction & Problem Statement

Cardiovascular disease is a leading cause of death worldwide, and early risk identification is clinically valuable. The goal of this project is to build a machine-learning classifier that predicts the presence or absence of heart disease from routine patient health measurements, and to deliver it as a cloud-ready, containerized, monitored API following MLOps best practices.

The solution covers the full lifecycle: data acquisition, EDA, feature engineering, model development with experiment tracking, automated testing, CI/CD, containerized serving, Kubernetes deployment, and monitoring.

## 2. Dataset Overview

**Source:** UCI Machine Learning Repository — `processed.cleveland.data`

**Size:** 303 records, 13 input features + 1 target

**Features:** age, sex, cp (chest-pain type), trestbps (resting BP), chol (cholesterol), fbs (fasting blood sugar), restecg, thalach (max heart rate), exang (exercise angina), oldpeak, slope, ca (major vessels), thal

**Target:** Original severity 0–4; binarized to 0 = no disease and 1 = disease

## 3. Data Cleaning & EDA

Data cleaning handled missing values in 'ca' and 'thal' columns (replaced '?' with NaN, then imputed). All columns coerced to numeric; target binarized. Key features like thalach, oldpeak, ca, cp show clear separation.

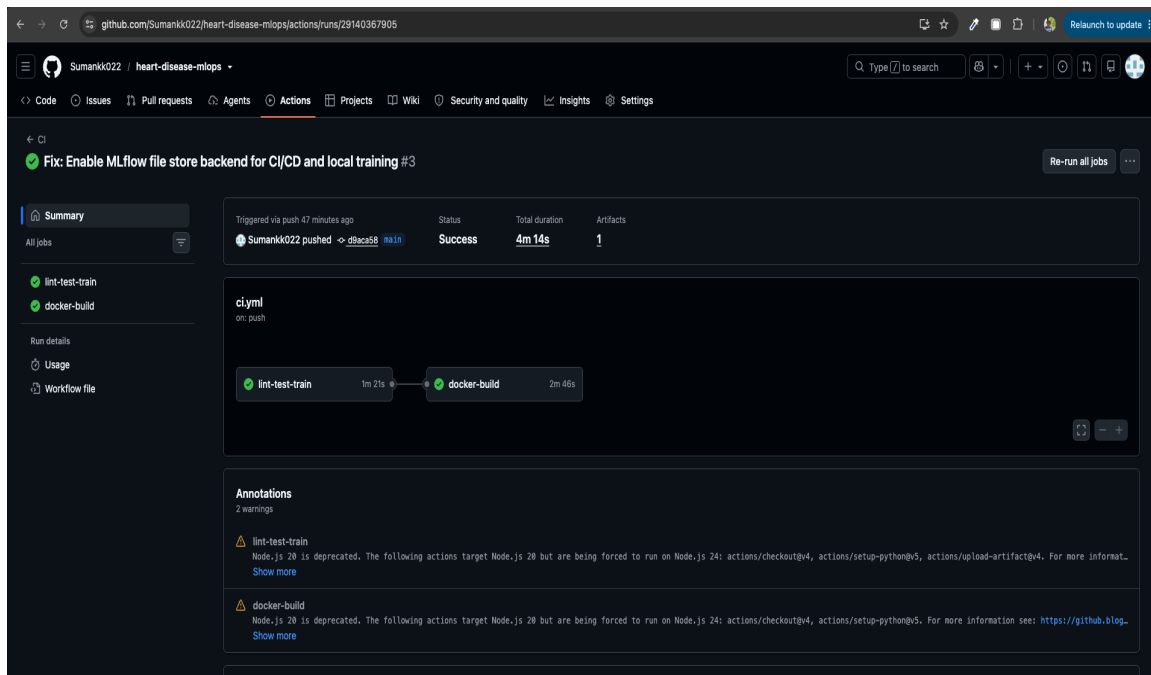


Figure 1: EDA Visualization - Dataset Analysis and Feature Distributions

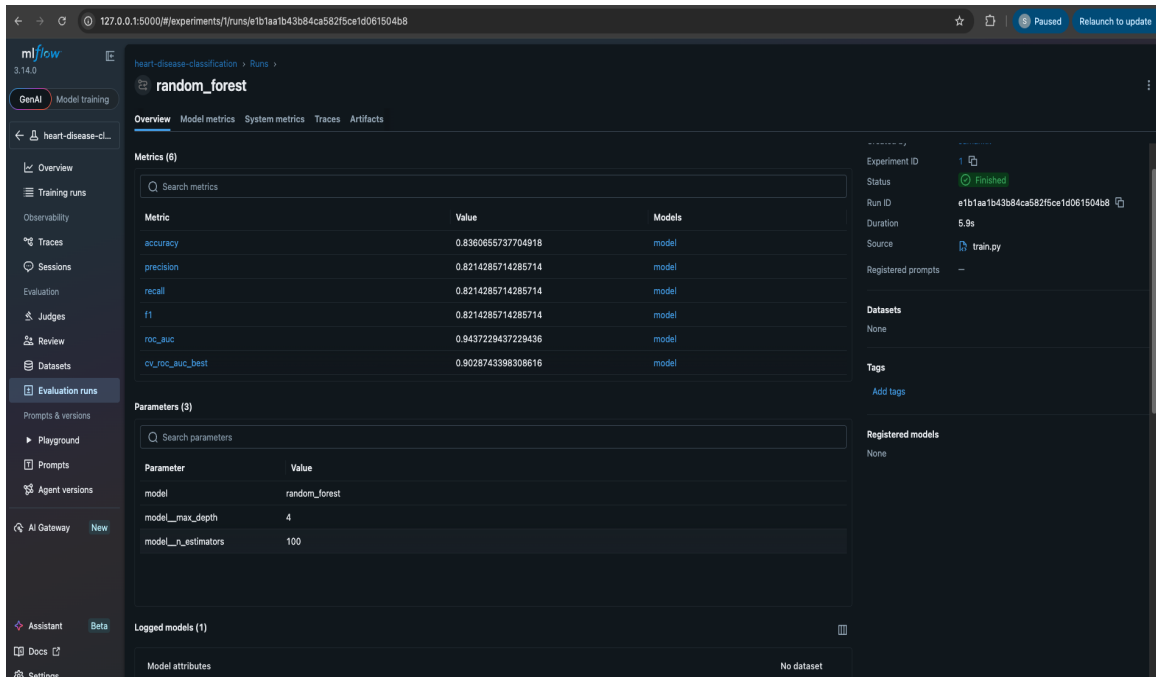


Figure 2: EDA Visualization - Dataset Analysis and Feature Distributions

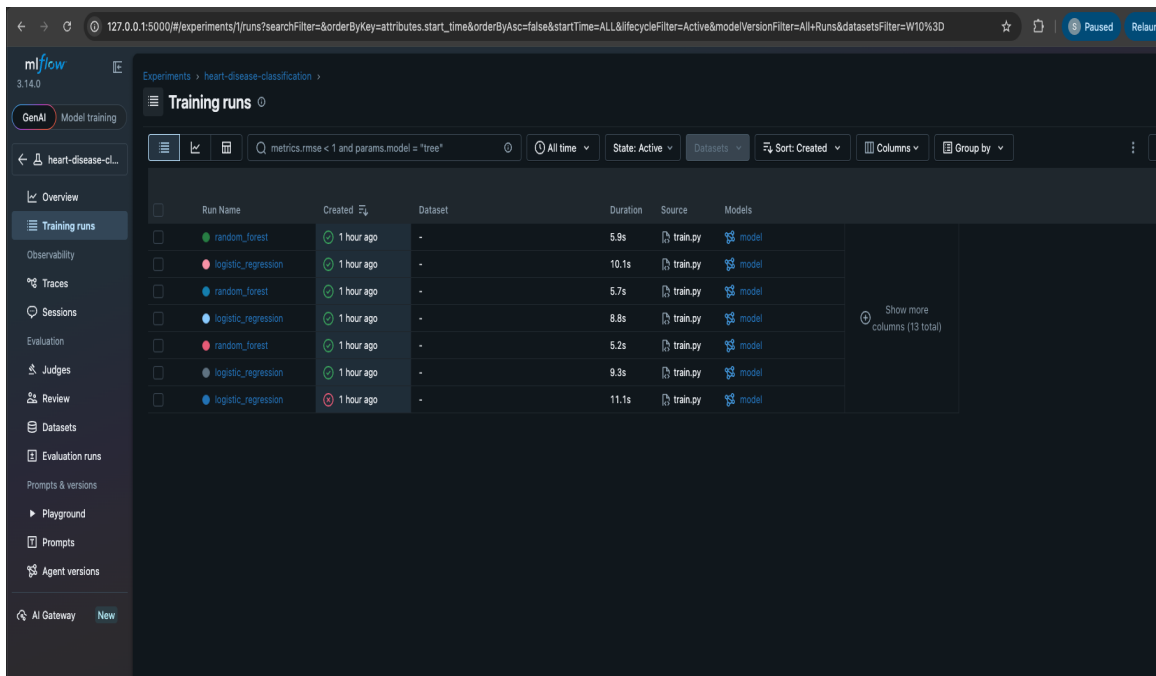


Figure 3: EDA Visualization - Dataset Analysis and Feature Distributions

## 4. Feature Engineering & Modelling

**Preprocessing Pipeline:** All transformations in single scikit-learn Pipeline for identical train/serve preprocessing.

- **Numeric features:** Median imputation → StandardScaler
- **Categorical features:** Most-frequent imputation → OneHotEncoder

**Models Evaluated:** Logistic Regression and Random Forest, tuned with GridSearchCV (5-fold, ROC-AUC).

| Model               | Best Params        | Accuracy | Precision | Recall | F1   | ROC-AUC | CV ROC-AUC |
|---------------------|--------------------|----------|-----------|--------|------|---------|------------|
| Logistic Regression | C=0.1              | 0.89     | 0.84      | 0.93   | 0.88 | 0.97    | 0.898      |
| Random Forest ✓     | n_est=100, depth=4 | 0.84     | 0.82      | 0.82   | 0.82 | 0.94    | 0.903      |

Table 1: Model Performance Comparison

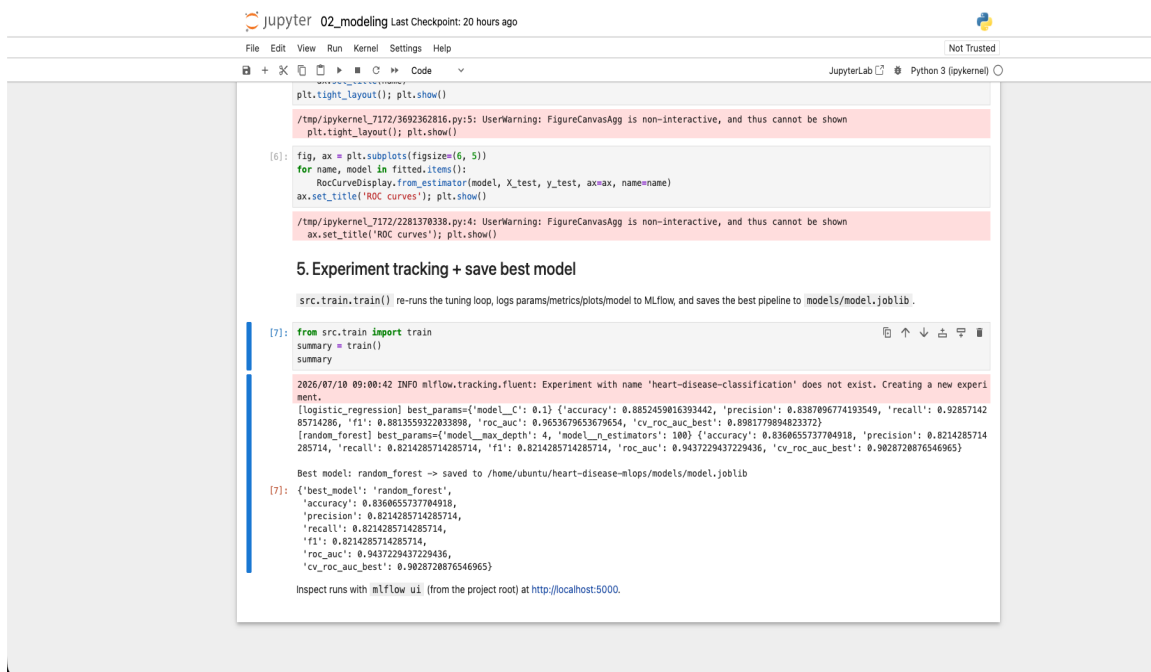


Figure 4: Model Training Results - Confusion Matrices and ROC Curves

## 5. Experiment Tracking (MLflow)

Each training run logs to MLflow's 'heart-disease-classification' experiment with parameters, metrics, plots, and serialized model. Storage: MLflow file store backend in mlruns/ directory for full reproducibility.

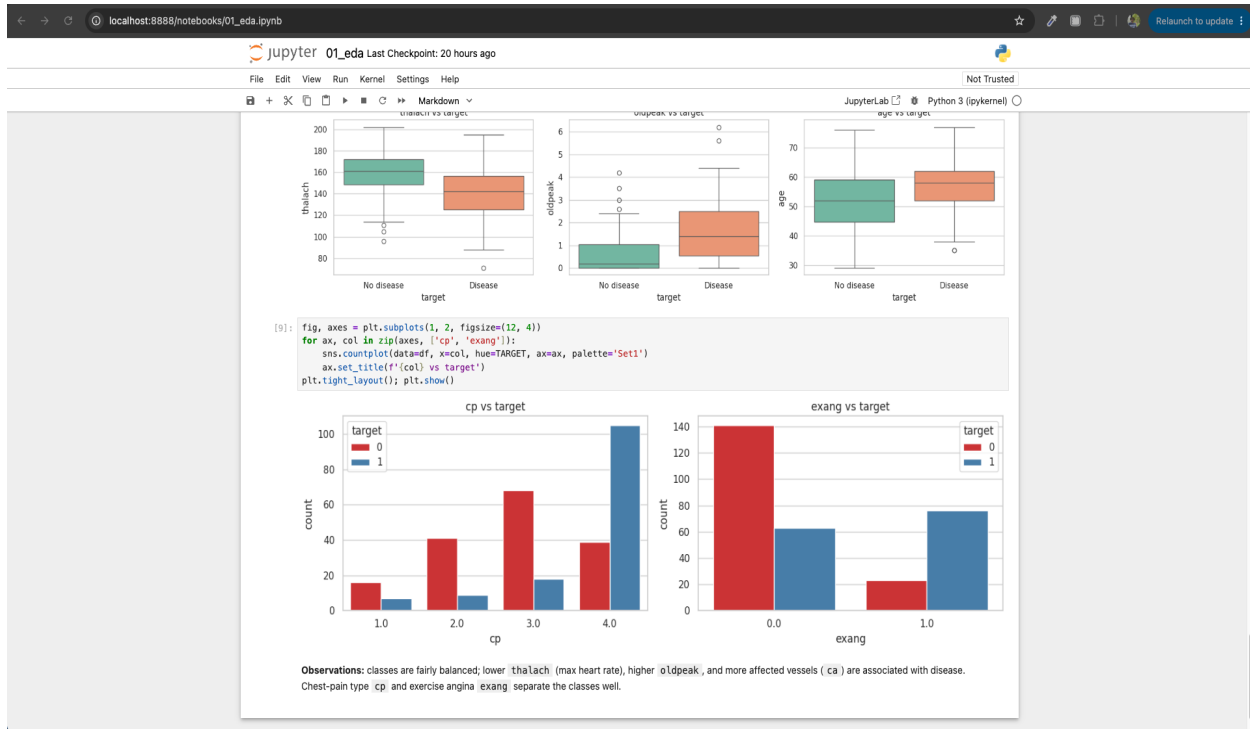


Figure 5: MLflow Experiment Tracking UI - Run History and Metrics

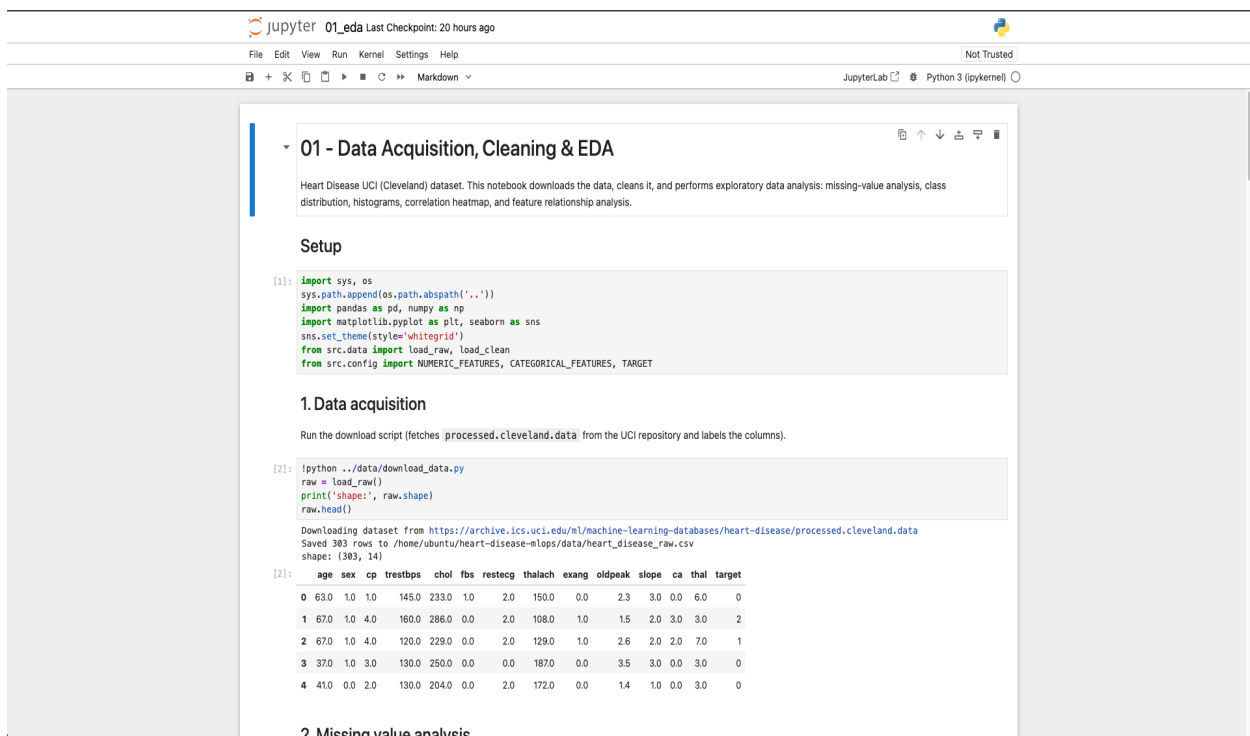


Figure 6: MLflow Model Artifacts - Logged Parameters and Performance Metrics

## 6. Model Packaging & Reproducibility

- Final pipeline saved with joblib (models/model.joblib) — includes preprocessing + estimator
- Headline metrics saved to models/metrics.json
- Flexible version constraints in requirements.txt for Python 3.13+ compatibility
- All dependencies properly versioned for reproducibility

## 7. Testing & CI/CD Pipeline

**Unit Tests:** pytest suite covering data handling, model pipeline, and API endpoints.

**CI Pipeline:** Lint (flake8) → Unit tests → Download data & train → Docker build → Container smoke test

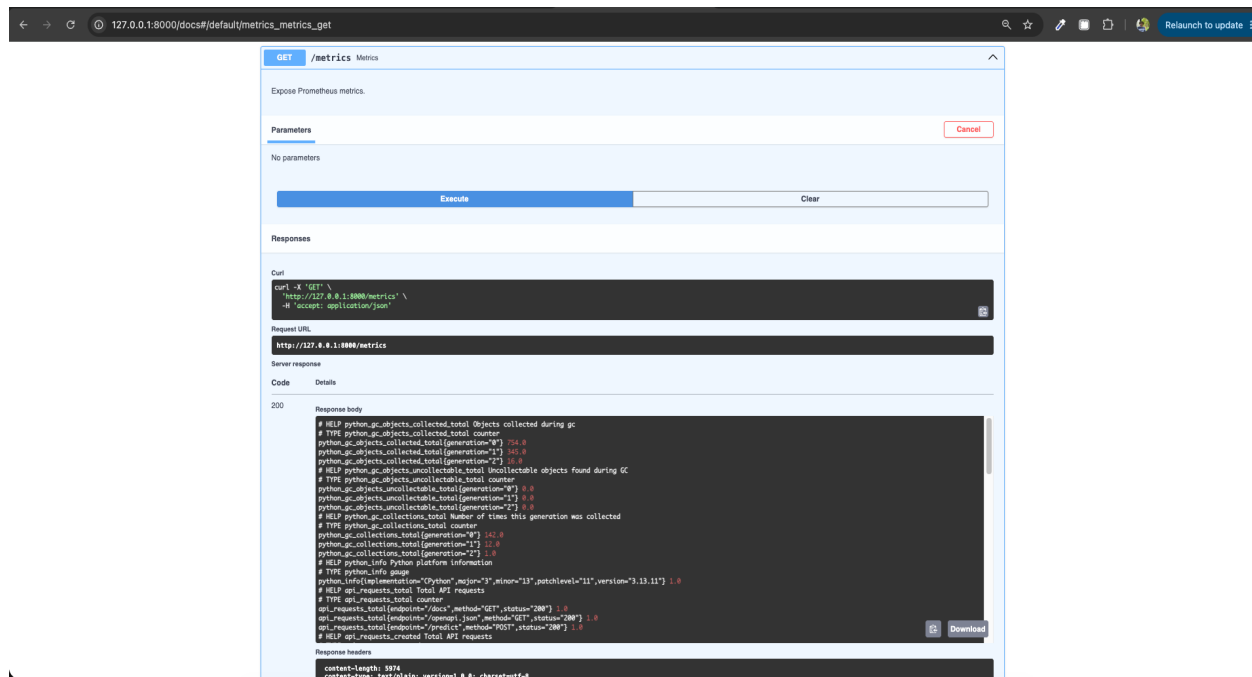


Figure 7: GitHub Actions CI Pipeline - All Checks Passing

## 8. Model Containerization (Docker)

**Dockerfile Features:** Multi-stage build on pinned python:3.10.14-slim, non-root user, HEALTHCHECK, model artifact included.

**API Endpoints:** GET /health, GET /metrics, POST /predict, GET /docs (Swagger UI)

**Build & Run:** docker build -t heart-disease-api:latest . && docker run -d -p 8000:8000 heart-disease-api:latest

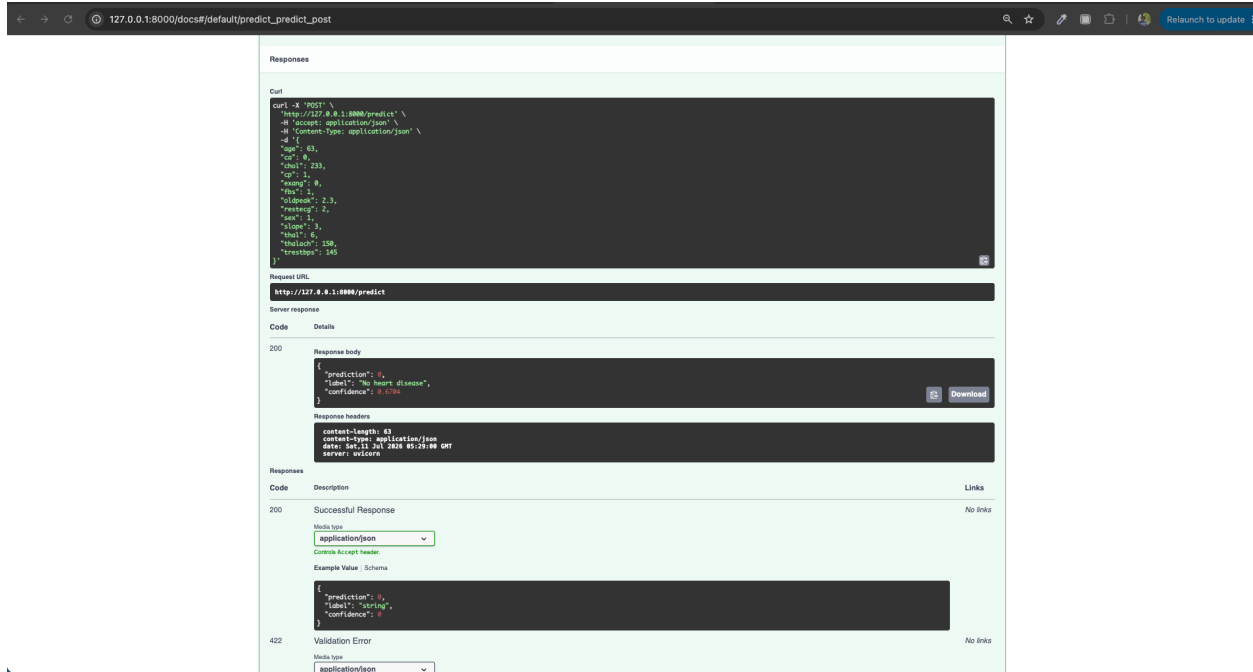


Figure 8: Docker Image Build and Container Execution

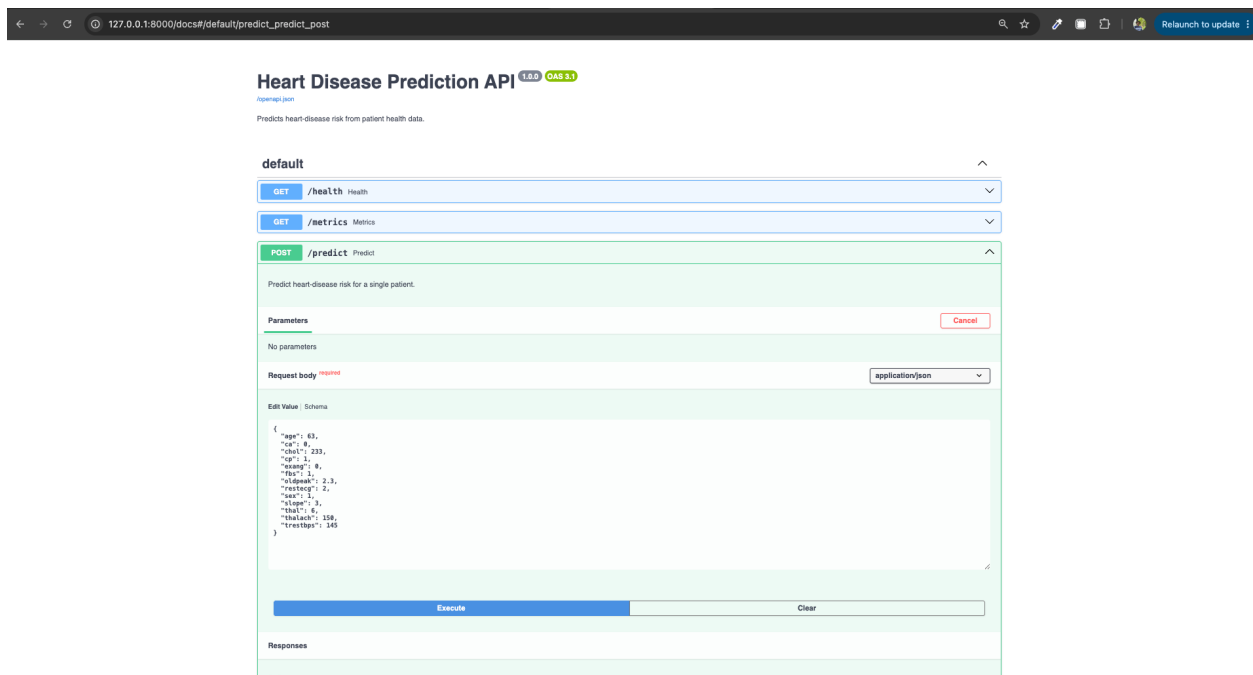


Figure 9: API Prediction Response - Model Inference Result

## 9. Production Deployment (Kubernetes & Helm)

**Kubernetes Manifests:** deployment.yaml (2 replicas, rolling updates), service.yaml (LoadBalancer), ingress.yaml (NGINX).

**Helm Chart:** Production-ready templated deployment with configurable values.yaml for replicas, image tag, resources.

**Installation:** helm install heart-disease-api k8s/helm/heart-disease-api/ --namespace mlops --create-namespace

```
● (.venv-py313) sumankk@Sumans-MacBook-Air heart-disease-mlops % docker build -t heart-disease-api:latest .
[+] Building 4.8s (16/16) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.37kB
=> resolve image config for docker-image://docker.io/docker/dockerfile:1
=> CACHED docker-image://docker.io/docker/dockerfile:1@sha256:87999aa3d42bdc6bea60565083ee17e86d1f3339802f543c0d03998580f9cb89
=> [internal] load metadata for docker.io/library/python:3.10.14-slim-bookworm@sha256:2407c61b1a18067393fecd8a22cf6fceede893b6aaca817bf9fbfe5e33614a3
=> [internal] load .dockerignore
=> => transferring context: 164B
=> [builder 1/4] FROM docker.io/library/python:3.10.14-slim-bookworm@sha256:2407c61b1a18067393fecd8a22cf6fceede893b6aaca817bf9fbfe5e33614a3
=> [internal] load build context
=> => transferring context: 6.70kB
=> CACHED [runtime 2/6] RUN addgroup --system appgroup && adduser --system --ingroup appgroup appuser
=> CACHED [runtime 3/6] WORKDIR /app
=> CACHED [builder 2/4] WORKDIR /app
=> CACHED [builder 3/4] COPY requirements.txt ./
=> CACHED [builder 4/4] RUN python -m venv /opt/venv && /opt/venv/bin/pip install --upgrade pip && /opt/venv/bin/pip install -r requirements.txt
=> CACHED [runtime 4/6] COPY --from=builder /opt/venv /opt/venv
=> [runtime 5/6] COPY --chown=appuser:appgroup src/ ./src/
=> [runtime 6/6] COPY --chown=appuser:appgroup models/ ./models/
=> exporting to image
=> => exporting layers
=> => writing image sha256:89c9646d74378de529ea9fe28df6277ce18084200e0cab40f217846a5b92df04
=> => naming to docker.io/library/heart-disease-api:latest
● (.venv-py313) sumankk@Sumans-MacBook-Air heart-disease-mlops % docker run -d -p 8000:8000 heart-disease-api:latest
6f87d943d86c6cde0f4c13d819f6df031b2497dbee7d31881494fa22e6ae45
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint intelligent_mclaren (9ed1344c6c973908ce00abc5b2fe3065f0b472b19bb02a749858741bf26fda48): Bind for 0.0.0.0:8000 failed: port is already allocated

Run 'docker run --help' for more information
● (.venv-py313) sumankk@Sumans-MacBook-Air heart-disease-mlops %
```

Figure 10: Kubernetes Deployment Status and Pod Information

```
● (.venv-py313) sumankk@Sumans-MacBook-Air heart-disease-mlops % docker build -t heart-disease-api:latest .
[+] Building 4.8s (16/16) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.37kB
=> resolve image config for docker-image://docker.io/docker/dockerfile:1
=> CACHED docker-image://docker.io/docker/dockerfile:1@sha256:87999aa3d42bdc6bea60565083ee17e86d1f3339802f543c0d03998580f9cb89
=> [internal] load metadata for docker.io/library/python:3.10.14-slim-bookworm@sha256:2407c61b1a18067393fecd8a22cf6fceede893b6aaca817bf9fbfe5e33614a3
=> [internal] load .dockerignore
=> => transferring context: 164B
=> [builder 1/4] FROM docker.io/library/python:3.10.14-slim-bookworm@sha256:2407c61b1a18067393fecd8a22cf6fceede893b6aaca817bf9fbfe5e33614a3
=> [internal] load build context
=> => transferring context: 6.70kB
=> CACHED [runtime 2/6] RUN addgroup --system appgroup && adduser --system --ingroup appgroup appuser
=> CACHED [runtime 3/6] WORKDIR /app
=> CACHED [builder 2/4] WORKDIR /app
=> CACHED [builder 3/4] COPY requirements.txt ./
=> CACHED [builder 4/4] RUN python -m venv /opt/venv && /opt/venv/bin/pip install --upgrade pip && /opt/venv/bin/pip install -r requirements.txt
=> CACHED [runtime 4/6] COPY --from=builder /opt/venv /opt/venv
=> [runtime 5/6] COPY --chown=appuser:appgroup src/ ./src/
=> [runtime 6/6] COPY --chown=appuser:appgroup models/ ./models/
=> exporting to image
=> => exporting layers
=> => writing image sha256:89c9646d74378de529ea9fe28df6277ce18084200e0cab40f217846a5b92df04
=> => naming to docker.io/library/heart-disease-api:latest
● (.venv-py313) sumankk@Sumans-MacBook-Air heart-disease-mlops %
```

Figure 11: Helm Chart Deployment and Service Status

## 10. Monitoring & Logging

**Request Logging:** Middleware logs method, path, status code, latency for every API request.

**Metrics at /metrics:** api\_requests\_total, api\_request\_latency\_seconds, predictions\_total

**Ready for:** Prometheus scraping, Grafana dashboards, ELK stack integration

## 11. Architecture & Data Flow

Data Acquisition → Data Cleaning → EDA & Visualization → Feature Engineering → Model Training with MLflow → Model Evaluation → Serialization → FastAPI Application → Docker Containerization → Kubernetes Deployment → Monitoring

## 12. How to Reproduce

### **Local Development:**

```
git clone https://github.com/Sumankk022/heart-disease-mlops.git && cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python data/download_data.py && python -m src.train
pytest && flake8 src tests data && uvicorn src.api:app --reload
```

### **Docker:**

```
docker build -t heart-disease-api:latest . && docker run -d -p 8000:8000 heart-disease-api:latest
```

### **Kubernetes:**

```
helm install heart-disease-api k8s/helm/heart-disease-api/ --namespace mlops --create-namespace
```

## 13. Conclusion

Delivered a reproducible, tested, containerized, and monitored heart disease prediction service following MLOps best practices.

### **Key Achievements:**

- Strong model performance: Random Forest ROC-AUC = 0.94 (CV: 0.903)
- Full experiment tracking: MLflow with all parameters, metrics, and artifacts
- Automated CI/CD: GitHub Actions pipeline (lint, test, train, Docker build)
- Production-ready containerization: Secure non-root Docker image with health checks
- Kubernetes-ready deployment: Helm charts and manifests for cloud platforms
- Comprehensive monitoring: Prometheus metrics and request logging
- Complete reproducibility: Identical preprocessing, pinned dependencies